

Core Principle: Unlike **for loops** (which are often $O(n)$ by default), **while loops** and **if statements** require **detailed analysis** to determine time complexity - study the condition and trace iterations.

1. Historical Context of Loops

Old Languages (Pascal-style)

```
FOR i:= 1 TO n DO  
  statements
```

- **Linear increment** by default ($i=1,2,3,\dots,n$) $\rightarrow O(n)$
- **STEP** keyword for custom increments (e.g., STEP 2 $\rightarrow i=1,3,5,\dots$)
- Still **linear growth** regardless of step size

Modern C Language Loops

- **for, while, do-while** - functionally equivalent
 - **do-while: Post-tested** (executes at least once)
 - **while: Pre-tested** (may execute 0 times)
-

2. Key Analysis Rule for While Loops

MANDATORY: Trace the **termination condition** - count iterations until condition becomes **false**.

"You have to study this loop and find out when it is going to stop and how many times it is going to iterate."

Never assume $O(n)$ - could be $O(\log n)$, $O(\sqrt{n})$, or $O(1)$.

3. While Loop Examples (Step-by-Step Analysis)

Example 1: Linear While Loop ($O(n)$)

```
i = 0;
while(i < n) {
    // statement (executes n times)
    i++;
}
```

Trace:

- Condition checked: **$n+1$** times
- Body executes: **n** times
- **Total:** $3n + 2 \rightarrow O(n)$

Equivalent for loop:

```
for(i=0; i<n; i++) { ... }
```

Same analysis: **ignore constants**, focus on **degree** (n).

Example 2: Geometric While Loop ($O(\log n)$)

```
a = 1;
while(a < b) {
    // statement
    a = a * 2;
}
```

Trace:

1. $a=1$ (2^0)
2. $a=2$ (2^1)
3. $a=4$ (2^2)
- 4....
4. $a=2^k \geq b \rightarrow$ **stops**

Solve: $2^k = b \rightarrow k = \log_2 b \rightarrow \mathbf{O(\log n)}$ (let $b=n$)

Equivalent for loop:

```
for(a=1; a<b; a*=2) { ... }
```

Example 3: Square Root Loop ($O(\sqrt{n})$)

```
i=1, k=1;
while(k < n) {
    k = k + i; // k += i
    i++;
}
```

Trace (assume m iterations):

```
Iteration 1: k=1+1=2, i=2
Iteration 2: k=2+2=4, i=3
Iteration 3: k=4+3=7, i=4...
Iteration m:  $k \approx m(m+1)/2 \geq n$ 
```

Solve: $m^2/2 \approx n \rightarrow m \approx \sqrt{2n} \rightarrow \mathbf{O(\sqrt{n})}$

Example 4: GCD Algorithm ($O(n)$ worst case)

```
while(m!= n) {
    if(m > n) m = m - n;
    else n = n - m;
}
```

Cases:

- **Best:** $m=n$ initially $\rightarrow \mathbf{0}$ iterations
- **Typical:** Few subtractions
- **Worst:** $m=16, n=2 \rightarrow \mathbf{7}$ iterations $(16/2) \rightarrow \mathbf{O(n)}$

4. If Statements Impact

Conditional Execution

```
if(n < 5) {  
    print(n);    // O(1)  
} else {  
    for(i=0; i<n; i++) { // O(n)...  
    }  
}
```

Analysis:

- **$n < 5$: Best case $O(1)$**
- **$n \geq 5$: Worst case $O(n)$**
- **if doesn't guarantee** different cases - **depends on condition**

Counter-example: `if(n > 5) { loop n times }` → **always $O(n)$** regardless of if

5. Universal Truth

Any while loop logic → convertible to for loop in C

```
while(condition) {  
    body;  
    update;  
}
```

≡

```
for(initialize; condition; update) {  
    body;  
}
```

Analysis method remains identical.